Comprehensive Notes: CSRF (Cross-Site Request Forgery) in Spring Boot

1. Theory: What is CSRF?

CSRF stands for **Cross-Site Request Forgery**. It is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated.

The Problem Scenario

1. **Authentication:** A user logs into a secure application (e.g., a banking site or your Spring Boot app). The server issues a **Session ID** (cookie) to keep the user logged in.
2. **The Vulnerability:** The user visits a malicious website in another tab while still logged into the secure application.
3. **The Attack:** The malicious website contains a script that automatically sends a request (e.g., "Transfer Money" or "Delete Account") to the secure application's server.
4. **The Result:** Because the browser automatically attaches the valid **Session ID** cookie to the request, the server thinks the request is legitimate and processes it.

The Solution: CSRF Token

To prevent this, Spring Security implements a **CSRF Token**.

- **State-Changing Requests:** Methods like `POST`, `PUT`, `DELETE` (which modify data) require a unique, unpredictable token.
- **Read-Only Requests:** Methods like `GET` generally do not require a CSRF token.
- **Mechanism:** The server generates a token. The client must send this token back with every state-changing request. If the token is missing or invalid, the server rejects the request.

2. Spring Security Default Behavior

- **CSRF Protection:** Enabled by default in Spring Security for session-based authentication.
- **GET Requests:** Allowed without a CSRF token.
- **POST/PUT/DELETE Requests:** Blocked (HTTP 403 Forbidden) if the CSRF token is not present in the request header or parameter.

3. Practical Implementation Example

The following code examples are based on the video transcript, cleaned up for clarity.

A. The Model (Student.java)

A simple POJO to represent data.

```java
public class Student {
    private int id;
    private String name;
    private int marks;

    // Parameterized Constructor
    public Student(int id, String name, int marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getMarks() { return marks; }
    public void setMarks(int marks) { this.marks = marks; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "', marks=" + marks + "}";
    }
}
```

B. The Controller (StudentController.java)

Handles the actual business logic (Fetching and Adding students).

```java
import java.util.ArrayList;
import java.util.List;

@RestController
public class StudentController {

    // Hardcoded list for demonstration (In production, use a Database)
    List<Student> students = new ArrayList<>(List.of(
        new Student(1, "Naven", 60),
        new Student(2, "Ken", 65)
    ));

    // 1. GET Request (Fetch Data)
    // Does NOT require CSRF Token by default
    @GetMapping("/students")
    public List<Student> getStudents() {
        return students;
    }

    // 2. POST Request (Add Data)
    // REQUIRES CSRF Token when Spring Security is enabled
    @PostMapping("/students")
    public Student addStudent(@RequestBody Student student) {
        students.add(student);
        return student;
    }
}
```

C. The CSRF Token Fetcher (CsrfTokenController.java)

Since we are testing with Postman (not a browser form), we need an endpoint to retrieve the current CSRF token to use in our headers.

```java
package com.example.demo;

import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import jakarta.servlet.http.HttpServletRequest; // Or javax.servlet.http depending on version

@RestController
public class CsrfTokenController {

    // Endpoint to fetch the CSRF Token
    @GetMapping("/csrf-token")
    public CsrfToken getCsrfToken(HttpServletRequest request) {
        // Spring Security stores the token in the request attribute "_csrf"
        return (CsrfToken) request.getAttribute("_csrf");
    }
}
```

4. Testing Scenarios & Outputs

Scenario 1: GET Request (Fetching Data)

Action: Send a GET request to http://localhost:8080/students with Basic Auth.**CSRF Token:** Not required.

- **Output (Success):**
-

[

{

"id": 1,

"name": "Naven",

"marks": 60

},

{

"id": 2,

"name": "Ken",

```
    "marks": 65
  }
]
```

Scenario 2: POST Request WITHOUT CSRF Token (Wrong Code)

Action: Send a `POST` request to `http://localhost:8080/students` with JSON body, but **without** the CSRF header.**Security:** Spring Security Enabled.

Output (Failure):

```
Status: 403 Forbidden
Body: {
    "error": "Forbidden",
    "message": "Access Denied",
    "status": 403
}
```

(Note: The transcript initially mentions "Unauthorized", but standard Spring Security CSRF failure is typically 403 Forbidden. The key takeaway is the request is rejected.)

Reason: Spring Security detected a state-changing request (POST) but did not find a valid CSRF token

Scenario 3: POST Request WITH CSRF Token (Correct Code)

Action:

1. First, call `GET /csrf-token` to get the value.
   - *Response:* `{"token": "a1b2c3d4...", "parameterName": "_csrf", "headerName": "X-CSRF-TOKEN"}`
2. Copy the token value.
3. Send `POST /students`.
4. **Header:** Add `X-CSRF-TOKEN` : `[Paste Token Value Here]`.
5. **Body:** `{"id": 3, "name": "Hush", "marks": 70}`.

- **Output (Success):**

```json
{
    "id": 3,
    "name": "Hush",
    "marks": 70
}
```

(Status: 200 OK)

Verification: If you run the `GET /students` requ again, the new student will appear in the list.

5. Alternative Solutions & Configuration

The video mentions that while CSRF tokens are standard for browser-based sessions, there are other ways to handle security depending on the architecture.

A. Disabling CSRF (For Stateless APIs)

If you are building a REST API that uses **JWT (JSON Web Tokens)** or is purely stateless, you often disable CSRF protection because you aren't relying on session cookies that browsers auto-submit.

Configuration Example (SecurityConfig.java):

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // Disables CSRF protection
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
```

```
    return http.build();
}
```

- **Warning:** Do not disable CSRF if you are using standard Session/Cookie authentication in a browser.

B. Stateless Sessions

Instead of a server-side session ID, use a stateless token (like JWT). Since there is no session cookie automatically sent by the browser, the CSRF attack vector is removed.

C. SameSite Cookie Attribute

Configure cookies to have the `SameSite=Strict` attribute. This tells the browser not to send cookies if the request originates from a different website (cross-site).

6. Summary Checklist

Feature	Description
Vulnerability	Malicious sites using your active session to perform actions.
Protection	**CSRF Token** (Unique per session/request).
Affected Methods	`POST`, `PUT`, `DELETE`, `PATCH` (State-changing).
Safe Methods	`GET`, `HEAD`, `OPTIONS` (Read-only).
Header Name	Typically `X-CSRF-TOKEN`.
Parameter Name	Typically `_csrf`.
Spring Default	CSRF Protection is **ON** by default.

7. Key Takeaways from Video

1. **Session ID vs. CSRF Token:** The Session ID proves *who* you are. The CSRF Token proves *you* intended to send *this specific request*.
2. **Browser vs. Postman:** Browsers handle CSRF tokens automatically in forms (via hidden input fields). Tools like Postman require you to manually fetch the token and add it to the Headers.

3. **Debugging:** If you get a 403 Error on a POST request despite having correct Login credentials, check if you are missing the CSRF Token.
4. **Source Code:** You can view the CSRF token in a browser by Right Click -> **View Page Source** and looking for `<input type="hidden" name="_csrf" value="...">`.